arXiv:2412.06769v3 [cs.CL] 3 Nov 2025

Training Large Language Models to Reason in a Continuous Latent Space

Shibo Hao[1,2,*], **Sainbayar Sukhbaatar**[1], **DiJia Su**[1], **Xian Li**[1], **Zhiting Hu**[2], **Jason Weston**[1], **Yuandong Tian**[1]

[1]FAIR at Meta, [2]UC San Diego
[*]Work done at Meta

Large language models (LLMs) are restricted to reason in the "language space", where they typically express the reasoning process with a chain-of-thought (CoT) to solve a complex reasoning problem. However, we argue that language space may not always be optimal for reasoning. For example, most word tokens primarily ensure textual coherence and are not essential for reasoning, while some critical tokens require complex planning and pose huge challenges to LLMs. To explore the potential of LLM reasoning in an unrestricted latent space instead of using natural language, we introduce a new paradigm COCONUT (Chain of Continuous Thought). We utilize the last hidden state of the LLM as a representation of the reasoning state (termed "continuous thought"). Rather than decoding this into a word token, we feed it back to the LLM as the subsequent input embedding directly in the continuous space. This latent reasoning paradigm leads to the emergence of an advanced reasoning pattern: the continuous thought can encode multiple alternative next reasoning steps, allowing the model to perform a breadth-first search (BFS) to solve the problem, rather than prematurely committing to a single deterministic path like CoT. COCONUT outperforms CoT on certain logical reasoning tasks that require substantial search during planning, and shows a better trade-off between accuracy and efficiency.

Last updated: November 4, 2025
Code: https://github.com/facebookresearch/coconut

Meta

1 Introduction

Large language models (LLMs) have demonstrated remarkable reasoning abilities, emerging from extensive pretraining on human languages (Dubey et al., 2024; Achiam et al., 2023). While next token prediction is an effective training objective, it imposes a fundamental constraint on the LLM as a reasoning machine: the explicit reasoning process of LLMs must be generated in word tokens. For example, a prevalent approach, known as chain-of-thought (CoT) reasoning (Wei et al., 2022), involves prompting or training LLMs to generate solutions step-by-step using natural language. However, this is in stark contrast to certain human cognition results. Neuroimaging studies have consistently shown that the language network – a set of brain regions responsible for language comprehension and production – remains largely inactive during various reasoning tasks (Amalric and Dehaene, 2019; Monti et al., 2012, 2007, 2009; Fedorenko et al., 2011). Further evidence indicates that human language is optimized for communication rather than reasoning (Fedorenko et al., 2024).

A significant issue arises when LLMs use language for reasoning: the amount of reasoning required for each particular token varies greatly, yet current LLM architectures allocate nearly the same computing budget for predicting every token. Most tokens in a reasoning chain are generated solely for fluency, contributing little to the actual reasoning process. By contrast, some critical tokens require complex planning and pose huge challenges to LLMs. While previous work has attempted to fix these problems by prompting LLMs to generate succinct reasoning chains (Madaan and Yazdanbakhsh, 2022), or performing additional reasoning before generating some critical tokens (Zelikman et al., 2024), these solutions remain constrained within the language space and do not solve the fundamental problems. On the contrary, it would be ideal for LLMs to have the freedom to reason without any language constraints, and then translate their findings into language only when necessary.

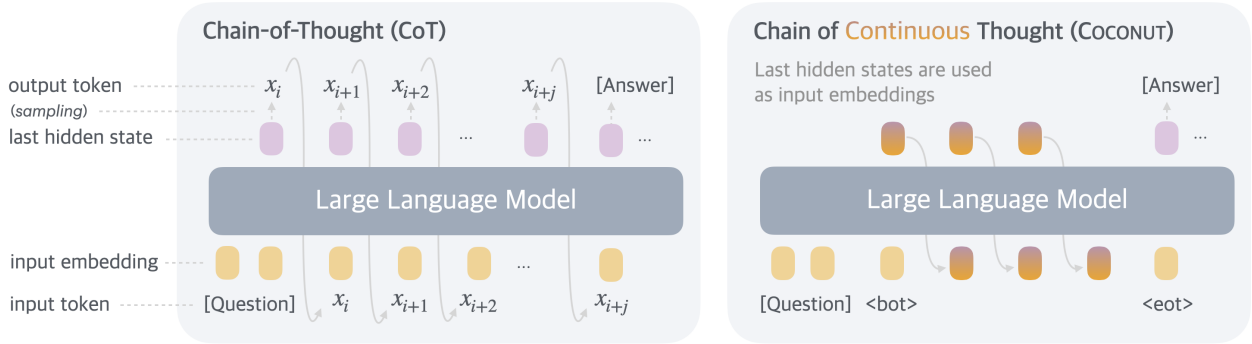

Figure 1 A comparison of Chain of Continuous Thought (Coconut) with Chain-of-Thought (CoT). In CoT, the model generates the reasoning process as a word token sequence (e.g., $[x_i, x_{i+1}, ..., x_{i+j}]$ in the figure). Coconut regards the last hidden state as a representation of the reasoning state (termed "continuous thought"), and directly uses it as the next input embedding. This allows the LLM to reason in an unrestricted latent space instead of a language space.

In this work we instead explore LLM reasoning in a latent space by introducing a novel paradigm, Coconut (Chain of Continuous Thought). It involves a simple modification to the traditional CoT process: instead of mapping between hidden states and language tokens using the language model head and embedding layer, Coconut directly feeds the last hidden state (a continuous thought) as the input embedding for the next token (Figure 1). This modification frees the reasoning from being within the language space, and the system can be optimized end-to-end by gradient descent, as continuous thoughts are fully differentiable. To enhance the training of latent reasoning, we employ a multi-stage training strategy inspired by Deng et al. (2024), which effectively utilizes language reasoning chains to guide the training process.

Interestingly, our proposed paradigm leads to an efficient reasoning pattern. Unlike language-based reasoning, continuous thoughts in Coconut can encode multiple potential next steps simultaneously, allowing for a reasoning process akin to breadth-first search (BFS). While the model may not initially make the correct decision, it can maintain many possible options within the continuous thoughts and progressively eliminate incorrect paths through reasoning, guided by some implicit value functions. This advanced reasoning mechanism surpasses traditional CoT, even though the model is not explicitly trained or instructed to operate in this manner, as seen in previous works (Yao et al., 2023; Hao et al., 2023).

Experimentally, Coconut successfully enhances the reasoning capabilities of LLMs. For math reasoning (GSM8k, Cobbe et al., 2021), using continuous thoughts is shown to be beneficial to reasoning accuracy, mirroring the effects of language reasoning chains. This indicates the potential to scale and solve increasingly challenging problems by chaining more continuous thoughts. On logical reasoning including ProntoQA (Saparov and He, 2022), and our newly proposed ProsQA (Section 4) which requires stronger planning ability, Coconut and some of its variants even surpasses language-based CoT methods, while generating significantly fewer tokens during inference. We believe that these findings underscore the potential of latent reasoning and could provide valuable insights for future research.

2 Related Work

Chain-of-thought (CoT) reasoning. We use the term chain-of-thought broadly to refer to methods that generate an intermediate reasoning process in language before outputting the final answer. This includes prompting LLMs (Wei et al., 2022; Khot et al., 2022; Zhou et al., 2022), or training LLMs to generate reasoning chains, either with supervised finetuning (Yue et al., 2023; Yu et al., 2023) or reinforcement learning (Wang et al., 2024; Havrilla et al., 2024; Shao et al., 2024; Yu et al., 2024a). Madaan and Yazdanbakhsh (2022) classified the tokens in CoT into symbols, patterns, and text, and proposed to guide the LLM to generate concise CoT based on analysis of their roles. Recent theoretical analyses have demonstrated the usefulness of CoT from the perspective of model expressivity (Feng et al., 2023; Merrill and Sabharwal, 2023; Li et al., 2024). By employing CoT, the effective depth of the transformer increases because the generated outputs are looped back to the input (Feng et al., 2023). These analyses, combined with the established effectiveness of CoT,

motivated our design of feeding the continuous thoughts back into the LLM as input embeddings. While CoT has proven effective for certain tasks, its autoregressive generation nature makes it challenging to mimic human reasoning on more complex problems (LeCun, 2022; Hao et al., 2023), which typically require planning and search. There are works that equip LLMs with explicit tree search algorithms (Xie et al., 2023; Yao et al., 2023; Hao et al., 2024), or train the LLM on search dynamics and trajectories (Lehnert et al., 2024; Gandhi et al., 2024; Su et al., 2024). In our analysis, we find that after removing the constraint of a language space, a new reasoning pattern similar to BFS emerges, even though the model is not explicitly trained in this way.

Latent reasoning in LLMs. Previous works mostly define latent reasoning in LLMs as the hidden computation in transformers (Yang et al., 2024; Biran et al., 2024). Yang et al. (2024) constructed a dataset of two-hop reasoning problems and discovered that it is possible to recover the intermediate variable from the hidden representations. Biran et al. (2024) further proposed to intervene the latent reasoning by "back-patching" the hidden representation. Shalev et al. (2024) discovered parallel latent reasoning paths in LLMs. Another line of work has discovered that, even if the model generates a CoT to reason, the model may actually utilize a different latent reasoning process. This phenomenon is known as the unfaithfulness of CoT reasoning (Wang et al., 2022; Turpin et al., 2024). To enhance the latent reasoning of LLMs, previous research proposed to augment it with additional tokens. Goyal et al. (2023) pretrained the model by randomly inserting a learnable `<pause>` tokens to the training corpus. This improves LLM's performance on a variety of tasks, especially when followed by supervised finetuning with `<pause>` tokens. On the other hand, Pfau et al. (2024) further explored the usage of filler tokens, e.g., "...", and concluded that they work well for highly parallelizable problems. However, Pfau et al. (2024) mentioned these methods do not extend the expressivity of the LLM like CoT; hence, they may not scale to more general and complex reasoning problems. Wang et al. (2023) proposed to predict a planning token as a discrete latent variable before generating the next reasoning step. Recently, it has also been found that one can "internalize" the CoT reasoning into latent reasoning in the transformer with knowledge distillation (Deng et al., 2023) or a special training curriculum which gradually shortens CoT (Deng et al., 2024). Yu et al. (2024b) also proposed to distill a model that can reason latently from data generated with complex reasoning algorithms. These training methods can be combined to our framework, and specifically, we find that breaking down the learning of continuous thoughts into multiple stages, inspired by iCoT (Deng et al., 2024), is very beneficial for the training. Other work explores alternative architectures for latent reasoning, including looped transformers (Giannou et al., 2023; Fan et al., 2024), diffusion models in sentence embedding space (Barrault et al., 2024). Different from these works, we focus on general multi-step reasoning tasks and aim to investigate the unique properties of latent reasoning in comparison to language space. In addition to reasoning tasks, Pham et al. (2023) also explored using continuous space for multi-agent communication. Building on Coconut, Zhu et al. (2025b) developed a theoretical framework demonstrating that continuous CoT can be more efficient than discrete CoT on certain tasks by encoding multiple reasoning paths in superposition states. Subsequently, Zhu et al. (2025a) analyzed the training dynamics to explain how such superposition emerges under the Coconut training objective.

3 Coconut: Chain of Continuous Thought

In this section, we introduce our new paradigm Coconut (Chain of Continuous Thought) for reasoning in an unconstrained latent space. We begin by introducing the background and notation we use for language models. For an input sequence $x = (x_1, ..., x_T)$, the standard large language model $\mathcal{M}$ can be described as:

$$H_t = \text{Transformer}(E_t)$$
$$\mathcal{M}(x_{t+1} \mid x_{\leq t}) = \text{softmax}(W h_t)$$

where $E_t = [e(x_1), e(x_2), ..., e(x_t)]$ is the sequence of token embeddings up to position t; $H_t \in \mathbb{R}^{t \times d}$ is the matrix of the last hidden states for all tokens up to position t; h_t is the last hidden state of position t, i.e., $h_t = H_t[t, :]$; $e(\cdot)$ is the token embedding function; W is the parameter of the language model head.

Method Overview. In the proposed Coconut method, the LLM switches between the "language mode" and "latent mode" (Figure 1). In language mode, the model operates as a standard language model, autoregressively

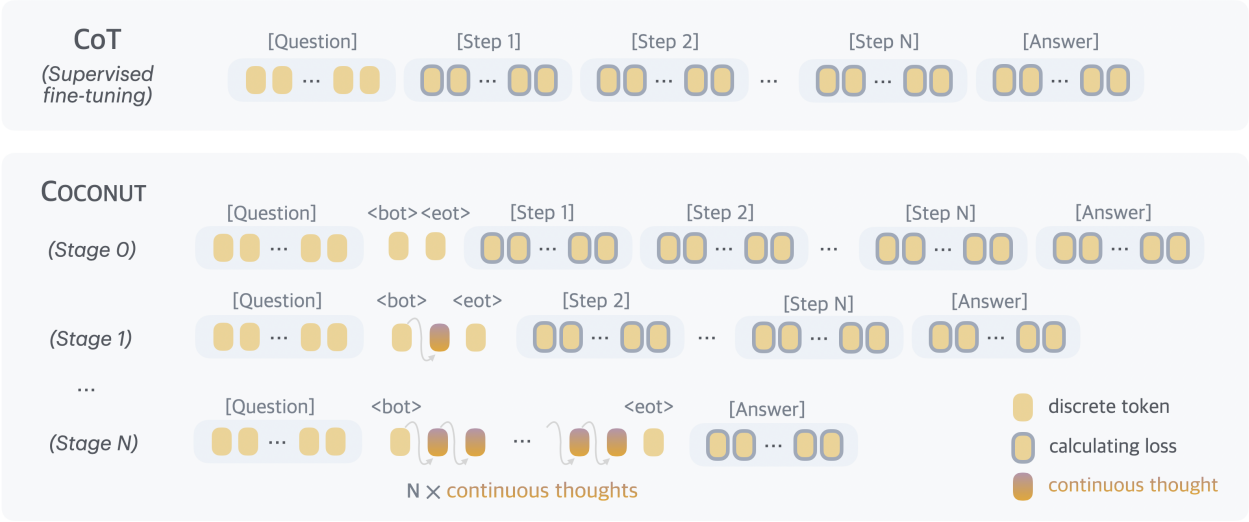

Figure 2 Training procedure of Chain of Continuous Thought (COCONUT). Given training data with language reasoning steps, at each training stage we integrate c additional continuous thoughts ($c = 1$ in this example), and remove one language reasoning step. The cross-entropy loss is then used on the remaining tokens after continuous thoughts.

generating the next token. In latent mode, it directly utilizes the last hidden state as the next input embedding. This last hidden state represents the current reasoning state, termed as a "continuous thought".

Special tokens <bot> and <eot> are employed to mark the beginning and end of the latent thought mode, respectively. As an example, we assume latent reasoning occurs between positions i and j, i.e., $x_i =$ <bot> and $x_j =$ <eot>. When the model is in the latent mode ($i < t < j$), we use the last hidden state from the previous token to replace the input embedding, i.e., $E_t = [e(x_1), e(x_2), ..., e(x_i), h_i, h_{i+1}, ..., h_{t-1}]$. After the latent mode finishes ($t \geq j$), the input reverts to using the token embedding, i.e., $E_t = [e(x_1), e(x_2), ..., e(x_i), h_i, h_{i+1}, ..., h_{j-1}, e(x_j), ..., e(x_t)]$. It is worth noting that the last hidden states have been processed by the final normalization layer, so they are not too large in magnitude. $\mathcal{M}(x_{t+1} \mid x_{\leq t})$ is not defined when $i < t < j$, since the latent thought is not intended to be mapped back to language space. However, $\text{softmax}(Wh_t)$ can still be calculated for probing purposes (see Section 5).

Training Procedure. In this work, we focus on a problem-solving setting where the model receives a question as input and is expected to generate an answer through a reasoning process. We leverage language CoT data to supervise continuous thought by implementing a multi-stage training curriculum inspired by Deng et al. (2024). As shown in Figure 2, in the initial stage, the model is trained on regular CoT instances. In the subsequent stages, at the k-th stage, the first k reasoning steps in the CoT are replaced with $k \times c$ continuous thoughts[1], where c is a hyperparameter controlling the number of latent thoughts replacing a single language reasoning step. Following Deng et al. (2024), we also reset the optimizer state when training stages switch. We insert <bot> and <eot> tokens (which are not counted towards c) to encapsulate the continuous thoughts.

During the training process, we optimize the normal negative log-likelihood loss, but mask the loss on questions and latent thoughts. It is important to note that the objective does **not** encourage the continuous thought to *compress the removed language thought*, but rather to *facilitate the prediction of future reasoning*. Therefore, it's possible for the LLM to learn more effective representations of reasoning steps compared to human language.

Training Details. Our proposed continuous thoughts are fully differentiable and allow for back-propagation. We perform $n + 1$ forward passes when n latent thoughts are scheduled in the current training stage, computing a new latent thought with each pass and finally conducting an additional forward pass to obtain a loss on the remaining text sequence. While we can save any repetitive computing by using a KV cache, the sequential nature of the multiple forward passes poses challenges for parallelism. Further optimizing the

[1]If a language reasoning chain is shorter than k steps, then all the language thoughts will be removed.

training efficiency of COCONUT remains an important direction for future research.

Inference Process. The inference process for COCONUT is analogous to standard language model decoding, except that in latent mode, we directly feed the last hidden state as the next input embedding. A challenge lies in determining when to switch between latent and language modes. As we focus on the problem-solving setting, we insert a `<bot>` token immediately following the question tokens. For `<eot>`, we consider two potential strategies: a) train a binary classifier on latent thoughts to enable the model to autonomously decide when to terminate the latent reasoning, or b) always pad the latent thoughts to a constant length. We found that both approaches work comparably well. Therefore, we use the second option in our experiment for simplicity, unless specified otherwise.

4 Continuous Space Enables Latent Tree Search

In this section, we provide a proof of concept of the advantage of continuous latent space reasoning. On ProsQA, a new dataset that requires extensive planning ability, COCONUT outperforms language space CoT reasoning. Interestingly, our analysis indicates that the continuous representation of reasoning can encode multiple alternative next reasoning steps. This allows the model to perform a breadth-first search (BFS) to solve the problem, instead of prematurely committing to a single deterministic path like language CoT.

We start by introducing the experimental setup (Section 4.1). By leveraging COCONUT's ability to switch between language and latent space reasoning, we are able to control the model to interpolate between fully latent reasoning and fully language reasoning and test their performance (Section 4.2). This also enables us to interpret the latent reasoning process as tree search (Section 4.3). Based on this perspective, we explain why latent reasoning can help LLMs make better decisions (Section 4.4).

4.1 Experimental Setup

Dataset. We introduce ProsQA (Proof with Search Question-Answering), a new logical reasoning dataset. A visualized example is shown in Figure 4. Each instance in ProsQA consists of a directed acyclic graph (DAG) of logical relationships between concepts, presented as natural language statements. The task requires models to determine logical relationships by finding valid paths through this graph, demanding sophisticated planning and search strategies. Unlike previous logical reasoning datasets like ProntoQA (Saparov and He, 2022), ProsQA's DAG structure introduces complex exploration paths, making it particularly challenging for models to identify the correct reasoning chain. More comprehensive details about the dataset construction and characteristics can be found in Appendix A.

Setup. We use a pre-trained GPT-2 model as the base model for all experiments. The learning rate is set to 1×10^{-4} while the effective batch size is 128. We train a COCONUT model following the training procedure in Section 3. Since the maximum reasoning steps in ProsQA is 6, we set the number of training stages to $N = 6$ in the training procedure. In each stage, we train the model for 5 epochs, and stay in the last stage until the 50 epochs. The checkpoint with the best accuracy in the last stage is used for evaluation. As reference, we report the performance of (1) *CoT*: the model is trained with CoT data, and during inference, the model will generate a complete reasoning chain to solve the problem. (2) *no-CoT*: the model is trained with only the question and answer pairs, without any reasoning steps. During inference, the model will output the final answer directly.

To understand the properties of latent and language reasoning space, *we manipulate the model to switch between fully latent reasoning and fully language reasoning*, by manually setting the position of the `<eot>` token during inference. When we enforce COCONUT to use k continuous thoughts, the model is expected to output the remaining reasoning chain in language, starting from the $k + 1$ step. In our experiments, we test variants of COCONUT on ProsQA with $k \in \{0, 1, 2, 3, 4, 5, 6\}$. Note that all these variants only differ in inference time while sharing the same model weights.

Metrics. We apply two sets of evaluation metrics. One of them is based on the correctness of the *final answer*, regardless of the reasoning process. It is also the main metric used in the later sections (Section 5.3). To enable fine-grained analysis on ProsQA, we define another metric on the *reasoning process*. We classify a reasoning chain into (1) **Correct Path**: The output is one of the shortest paths to the correct answer. (2)

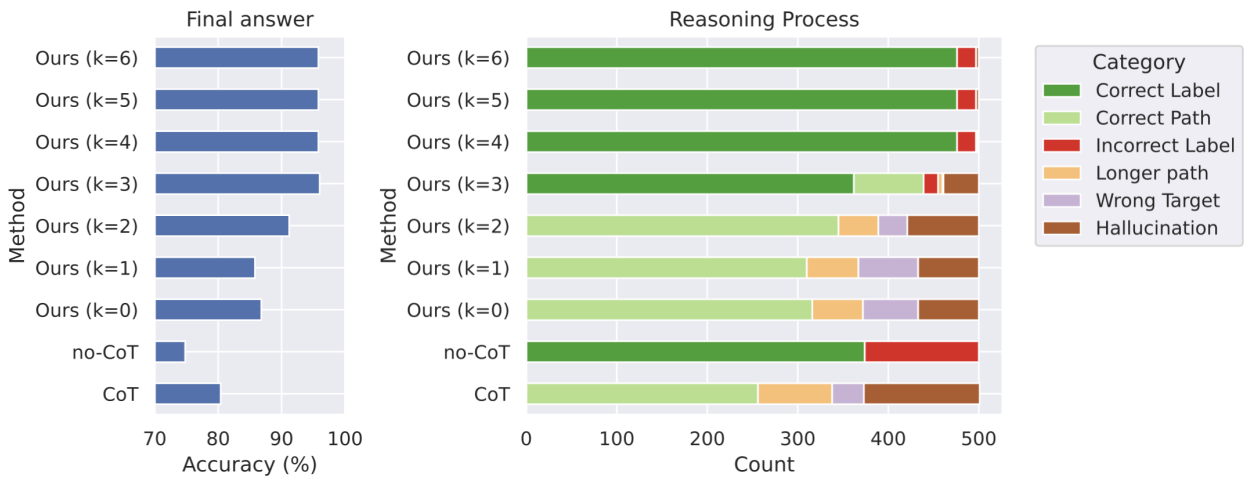

Figure 3 The accuracy of final answer (left) and reasoning process (right) of multiple variants of COCONUT and baselines on ProsQA.

Longer Path: A valid path that correctly answers the question but is longer than the shortest path. (3) **Hallucination**: The path includes nonexistent edges or is disconnected. (4) **Wrong Target**: A valid path in the graph, but the destination node is not the one being asked. These four categories naturally apply to the output from COCONUT ($k = 0$) and *CoT*, which generate the full path. For COCONUT with $k > 0$ that outputs only partial paths in language (with the initial steps in continuous reasoning), we classify the reasoning as a Correct Path *if a valid explanation can complete it*. Also, we define Longer Path and Wrong Target for partial paths similarly. If no valid explanation completes the path, it's classified as Hallucination. In *no-CoT* and COCONUT with larger k, the model may only output the final answer without any partial path, and it falls into (5) **Correct Label** or (6) **Incorrect Label**. These six categories cover all cases without overlap.

4.2 Overall Results

Figure 3 presents a comparative analysis of various reasoning methods evaluated on ProsQA. The model trained using *CoT* frequently hallucinates non-existent edges or outputs paths leading to incorrect targets, resulting in lower answer accuracy. In contrast, COCONUT, which leverages continuous space reasoning, demonstrates improved accuracy as it utilizes an increasing number of continuous thoughts. Additionally, the rate of correct reasoning processes (indicated by "Correct Label" and "Correct Path") significantly increases. At the same time, there is a notable reduction in instances of "Hallucination" and "Wrong Target," issues that typically emerge when the model makes mistakes early in the reasoning process.

An intuitive demonstration of the limitations of reasoning in language space is provided by the case study depicted in Figure 4. As shown, models operating in language space often fail to plan ahead or backtrack. Once they commit to an incorrect path, they either hallucinate unsupported edges or terminate with irrelevant conclusions. In contrast, latent reasoning avoids such premature commitments by enabling the model to iteratively refine its decisions across multiple reasoning steps. This flexibility allows the model to progressively eliminate incorrect options and converge on the correct answer, ultimately resulting in higher accuracy.

4.3 Interpreting the Latent Reasoning as Tree Search

To better understand COCONUT, we probe the latent reasoning process by forcing the model to explicitly generate language reasoning steps following intermediate continuous thoughts (Figure 5). Using the example presented in Figure 4, at the initial reasoning step, the model must select which immediate child node of "Alex" to consider next, specifically from the set {"lempus", "sterpus", "zhorpus", "grimpus"}. The distribution over these candidate next steps is visualized in Figure 5, left. In the subsequent reasoning step, these nodes expand further into an extended set of potential paths, including all grandchildren of "Alex" (Figure 5, right).

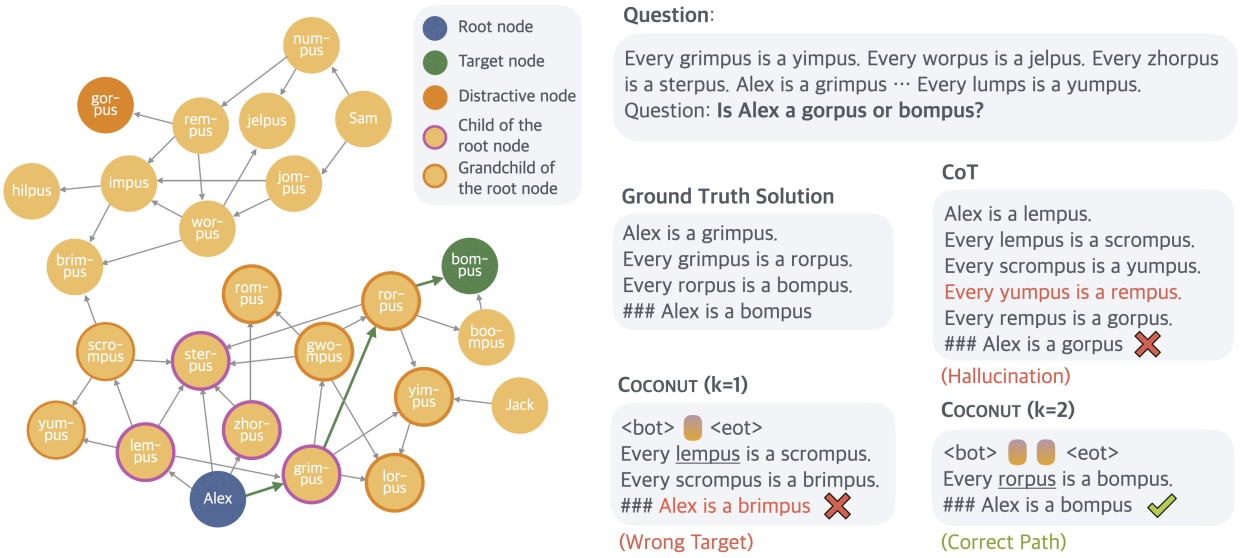

Figure 4 A case study of ProsQA. The model trained with *CoT* hallucinates an edge (*Every yumpus is a rempus*) after getting stuck in a dead end. Coconut (k=1) outputs a path that ends with an irrelevant node. Coconut (k=2) solves the problem correctly.

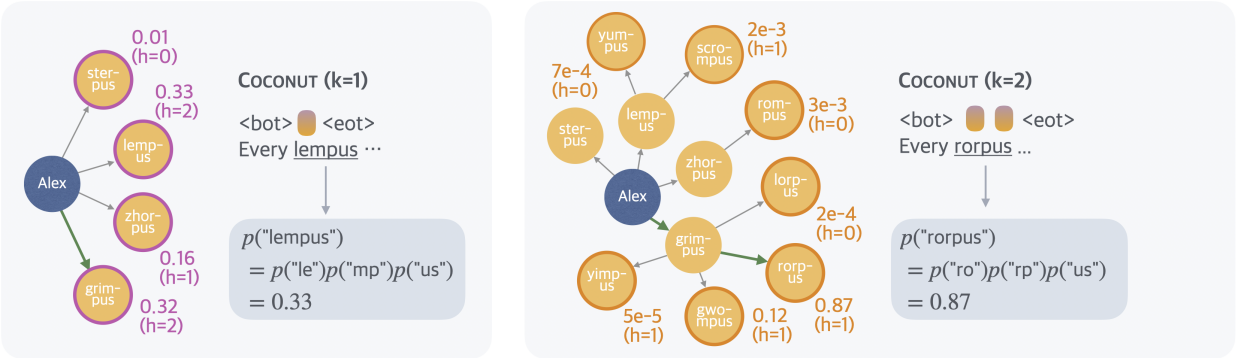

Figure 5 An illustration of the latent search trees. The example is the same test case as in Figure 4. The height of a node (denoted as h in the figure) is defined as the longest distance to any leaf nodes in the graph. We show the probability of the first concept predicted by the model following latent thoughts (e.g., "lempus" in the left figure). It is calculated as the multiplication of the probability of all tokens within the concept conditioned on previous context (omitted in the figure for brevity). This metric can be interpreted as an implicit value function estimated by the model, assessing the potential of each node leading to the correct answer.

We define the predicted probability of a concept following continuous thoughts as a value function (Figure 5), estimating each node's potential for reaching the correct target. Interestingly, the reasoning strategy employed by Coconut is not greedy search: while "lempus" initially has the highest value (0.33) at the first reasoning step (Figure 5, left), the model subsequently assigns the highest value (0.87) to "rorpus," a child of "grimpus," rather than following "lempus" (Figure 5, right). This characteristic resembles a breadth-first search (BFS) approach, contrasting sharply with the greedy decoding typical of traditional CoT methods. The inherent capability of continuous representations to encode multiple candidate paths enables the model to avoid making immediate deterministic decisions. Importantly, this tree search pattern is not limited to the illustrated example, but constitutes a fundamental mechanism underlying the consistent improvement observed with larger values of k in Coconut.

Figure 6 presents an analysis of the parallelism in the model's latent reasoning across the first and second thoughts. For the first thoughts (left panel), the cumulative values of the top-1, top-2, and top-3 candidate nodes are computed and plotted against their respective percentiles across the test set. The noticeable gaps between the three lines indicate that the model maintains significant diversity in its reasoning paths at this

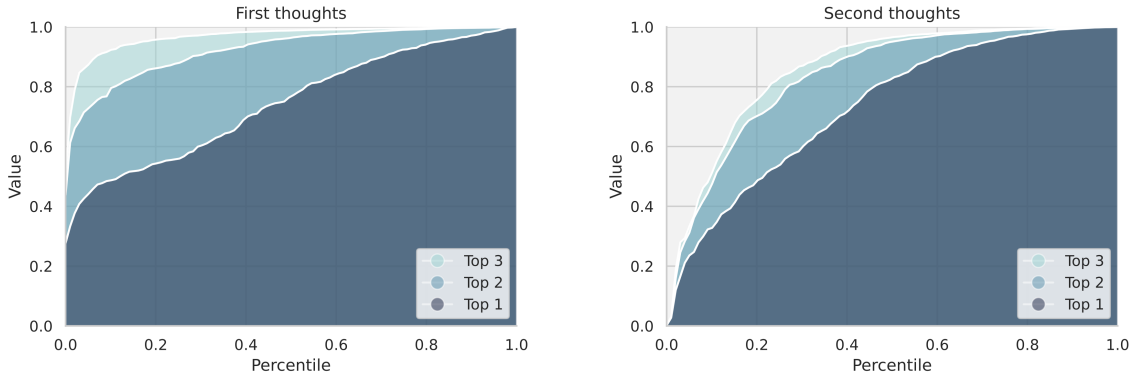

Figure 6 Analysis of parallelism in the first two steps of the latent tree search. The three curves in each panel depict the cumulative value of the top-1, top-2, and top-3 candidate nodes.

stage, suggesting a broad exploration of alternative possibilities. In contrast, the second thoughts (right panel) show a narrowing of these gaps. This trend suggests that the model transitions from parallel exploration to more focused reasoning in the second latent reasoning step, likely as it gains more certainty about the most promising paths.

4.4 Why is Latent Space Better for Planning?

Building upon the tree search perspective, we further examine why latent reasoning benefits planning tasks—specifically, why maintaining multiple candidate paths and postponing deterministic decisions enhances reasoning performance. Our hypothesis is that nodes explored in the early reasoning stages are inherently more challenging to evaluate accurately because they are farther from the final target nodes. In contrast, nodes positioned closer to potential targets, having fewer subsequent exploration possibilities, can be assessed accurately with higher confidence.

To systematically test this, we define the height of a node as its shortest distance to any leaf node and analyze the relationship between node height and the model's estimated value. Ideally, a correct node—one that can lead to the target node—should receive a high estimated value, whereas an incorrect node—one that cannot lead to the target node—should receive a low value. Empirical results across the test set (Figure 7) support our hypothesis: nodes with lower heights consistently receive more accurate and definitive probability evaluations. Conversely, nodes with greater heights exhibit more ambiguous evaluations, reflecting increased uncertainty.

These findings underscore the advantage of latent space reasoning. By delaying deterministic decisions and allowing exploration to proceed toward terminal states, latent reasoning significantly enhances the model's ability to differentiate correct paths from incorrect ones, thereby improving performance on complex, planning-intensive tasks compared to traditional greedy methods.

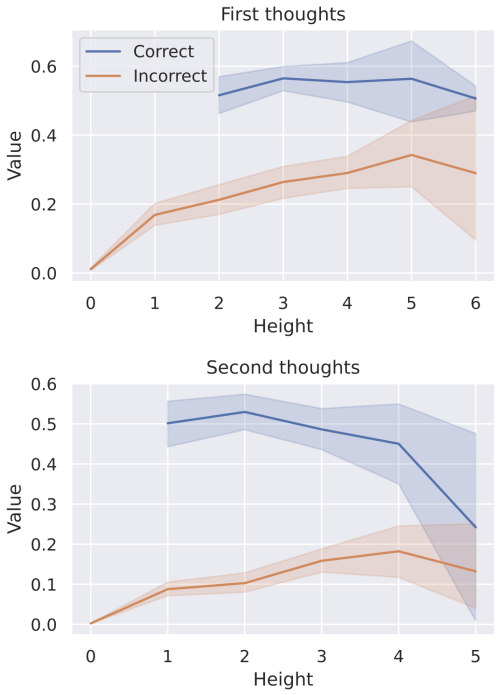

Figure 7 The correlation between the predicted value of correct/incorrect nodes and their heights.

5 Empirical Results with Coconut

After analyzing the promising parallel search pattern of Coconut, we validate the feasibility of LLM reasoning in a continuous latent space through more comprehensive experiments, highlighting its better reasoning efficiency over language space,

Method	GSM8k		ProntoQA		ProsQA	
	Acc. (%)	# Tokens	Acc. (%)	# Tokens	Acc. (%)	# Tokens
CoT	42.9 ±0.2	25.0	98.8 ±0.8	92.5	77.5 ±1.9	49.4
No-CoT	16.5 ±0.5	2.2	93.8 ±0.7	3.0	76.7 ±1.0	8.2
iCoT	30.0*	2.2	99.8 ±0.3	3.0	98.2 ±0.3	8.2
Pause Token	16.4 ±1.8	2.2	77.7 ±21.0	3.0	75.9 ±0.7	8.2
COCONUT (Ours)	34.1 ±1.5	8.2	99.8 ±0.2	9.0	97.0 ±0.3	14.2
- *w/o curriculum*	14.4 ±0.8	8.2	52.4 ±0.4	9.0	76.1 ±0.2	14.2
- *w/o thought*	21.6 ±0.5	2.3	99.9 ±0.1	3.0	95.5 ±1.1	8.2
- *pause as thought*	24.1 ±0.7	2.2	100.0 ±0.1	3.0	96.6 ±0.8	8.2

Table 1 Results on three datasets: GSM8k, ProntoQA and ProsQA. Higher accuracy indicates stronger reasoning ability, while generating fewer tokens indicates better efficiency. *The result is from Deng et al. (2024).

as well as its potential to enhance the model's expressivity with test-time scaling.

5.1 Experimental Setup

Math Reasoning. We use GSM8k (Cobbe et al., 2021) as the dataset for math reasoning. It consists of grade school-level math problems. To train the model, we use a synthetic dataset generated by Deng et al. (2023). We use two continuous thoughts for each reasoning step (i.e., $c = 2$). The model goes through 3 stages besides the initial stage. We then include an additional stage where still $3 \times c$ continuous thoughts are used as in the previous stage, but with all the remaining language reasoning chain removed. This handles the long-tail distribution of reasoning chains longer than 3 steps. We train the model for 6 epochs in the initial stage, and 3 epochs in each remaining stage.

Logical Reasoning. Logical reasoning involves the proper application of known conditions to prove or disprove a conclusion using logical rules. We use the ProntoQA (Saparov and He, 2022) dataset, and our newly proposed ProsQA dataset, which is more challenging due to more distracting branches. We use one continuous thought for every reasoning step (i.e., $c = 1$). The model goes through 6 training stages in addition to the initial stage, because the maximum number of reasoning steps is 6 in these two datasets. The model then fully reasons with continuous thoughts to solve the problems in the last stage. We train the model for 5 epochs per stage.

For all datasets, after the standard schedule, the model stays in the final training stage, until reaching 50 epochs. We select the checkpoint based on the accuracy on the validation set. For inference, we manually set the number of continuous thoughts to be consistent with their final training stage. We use greedy decoding for all experiments.

5.2 Baselines and Variants of Coconut

We consider the following baselines: (1) *CoT*, and (2) *No-CoT*, which were introduced in Section 4. (3) *iCoT* (Deng et al., 2024): The model is trained with language reasoning chains and follows a carefully designed schedule that "internalizes" CoT. As the training goes on, tokens at the beginning of the reasoning chain are gradually removed until only the answer remains. During inference, the model directly predicts the answer. (4) *Pause token* (Goyal et al., 2023): The model is trained using only the question and answer without a reasoning chain. However, different from *No-CoT*, special `<pause>` tokens are inserted between the question and answer, which provides the model with additional computational capacity to derive the answer. The number of `<pause>` tokens is set the same as continuous thoughts in COCONUT.

We also evaluate some variants of COCONUT: (1) *w/o curriculum*, which directly trains the model in the last stage. The model uses continuous thoughts to solve the whole problem. (2) *w/o thought*: We keep the multi-stage training, but don't add any continuous latent thoughts. While this is similar to *iCoT* in the high-level idea, the exact training schedule is set to be consistent with COCONUT, instead of *iCoT*, for a strict

comparison. (3) *Pause as thought*: We use special <pause> tokens to replace the continuous thoughts, and apply the same multi-stage training curriculum as COCONUT.

5.3 Results and Discussion

We show the overall results in Table 1. Using continuous thoughts effectively enhances LLM reasoning over the No-CoT baseline. For example, by using 6 continuous thoughts, COCONUT achieves 34.1% accuracy on GSM8k, which significantly outperforms *No-CoT* (16.5%). We highlight several key findings below.

"Chaining" continuous thoughts enhances reasoning. Language CoT proves to increase the effective depth of LLMs and enhance their expressiveness (Feng et al., 2023). Thus, generating more tokens serves as a way to inference-time scaling for reasoning (Guo et al., 2025; Snell et al., 2024). This desirable property holds naturally for COCONUT too. On GSM8k, COCONUT outperformed other architectures trained with similar strategies, including COCONUT (*pause as thought*) and COCONUT (*w/o thought*). Particularly, it surpasses the latest baseline *iCoT* (Deng et al., 2024), which requires a more carefully designed training schedule.

Additionally, we experimented with adjusting the hyperparameter c, which controls the number of latent thoughts corresponding to one language reasoning step (Figure 8, II). As we increased c from 0 to 1 to 2, the model's performance steadily improved.[2] This further validates the potential of continuous thoughts to scale up to harder problems. In two other synthetic tasks, we found that the variants of COCONUT (*w/o thoughts* or *pause as thought*), and the *iCoT* baseline also achieve impressive accuracy. This indicates that the model's computational capacity may not be the bottleneck in these tasks. In contrast, GSM8k involves more complex contextual understanding and modeling, placing higher demands on computational capability.

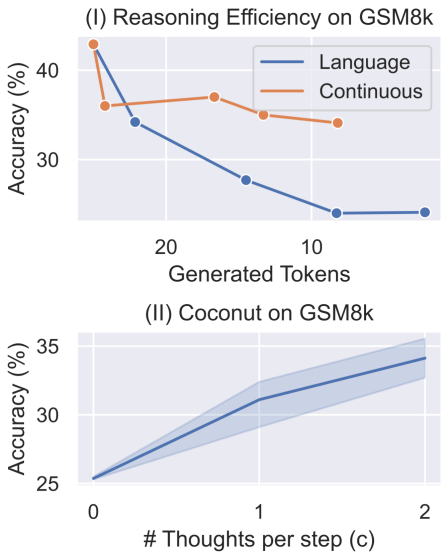

Figure 8 Efficiency comparison of reasoning space and COCONUT with different c.

Continuous thoughts are efficient representations of reasoning. Compared to traditional CoT, COCONUT generates fewer tokens while achieving higher accuracy on ProntoQA and ProsQA (Table 1). Although COCONUT does not surpass *CoT* on GSM8k, it offers a superior trade-off between reasoning efficiency and accuracy (Figure 8, I). To illustrate this, we train a series of CoT models that progressively "internalize" (Deng et al., 2024) the initial $m = \{0, 1, 2, 3, \text{ALL}\}$ reasoning steps, and plot their accuracy versus the number of generated tokens (labeled as "language" in the figure). These models quickly lose accuracy as more reasoning steps are skipped. In contrast, by applying COCONUT training strategy—replacing each language reasoning step with two continuous thoughts—the accuracy drop is substantially mitigated, maintaining higher performance even when fewer tokens are generated. Another interesting observation is that, when we decode the first continuous thought, it often corresponds to possible intermediate variables in the calculation (Figure 9). This also suggests that the continuous thoughts are more efficient representations of reasoning.

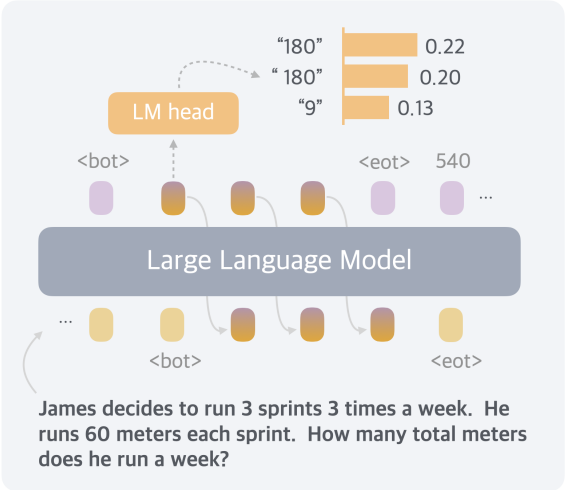

Figure 9 Decoding a continuous thought into language tokens in a math word problem. The decoded tokens correspond to intermediate variables that help solve the problem.

[2]We discuss the case of larger c in Appendix C.1.

The LLM still needs guidance to learn latent reasoning. In the ideal case, the model should learn the most effective continuous thoughts automatically through gradient descent on questions and answers (i.e., COCONUT *w/o curriculum*). However, from the experimental results, we found the models trained this way do not perform any better than no-CoT.

On the contrary, with the multi-stage curriculum, COCONUT is able to achieve top performance across various tasks. The multi-stage training also integrates well with pause tokens (COCONUT- *pause as thought*). Despite using the same architecture and similar multi-stage training objectives, we observed a small gap between the performance of *iCoT* and COCONUT (*w/o thoughts*). The finer-grained removal schedule (token by token) and a few other tricks in *iCoT* may ease the training process. We leave combining *iCoT* and COCONUT as future work. While the multi-stage training used for COCONUT has proven effective, further research is definitely needed to develop better and more general strategies for learning reasoning in latent space, especially without the supervision from language reasoning chains.

6 Conclusion

In this paper, we introduce COCONUT, a new paradigm for reasoning in continuous latent space. Experiments demonstrate that COCONUT effectively enhances LLM performance across a variety of reasoning tasks. Reasoning in latent space gives rise to advanced emergent behaviors, where continuous thoughts can represent multiple alternative next steps. This enables the model to perform BFS over possible reasoning paths, rather than prematurely committing to a single deterministic trajectory as in language space CoT reasoning. Further research is needed to refine and scale latent reasoning to pretraining, which could improve generalization across a broader range of reasoning challenges. We hope our findings will spark continued exploration into latent reasoning, ultimately advancing the development of more capable machine reasoning systems.